

Symmetric Encryption

Modern Ciphers

Computer Systems Security

2025/26

LEI - UM

Hugo Pacheco hpacheco@di.uminho.pt

One-Time Pad

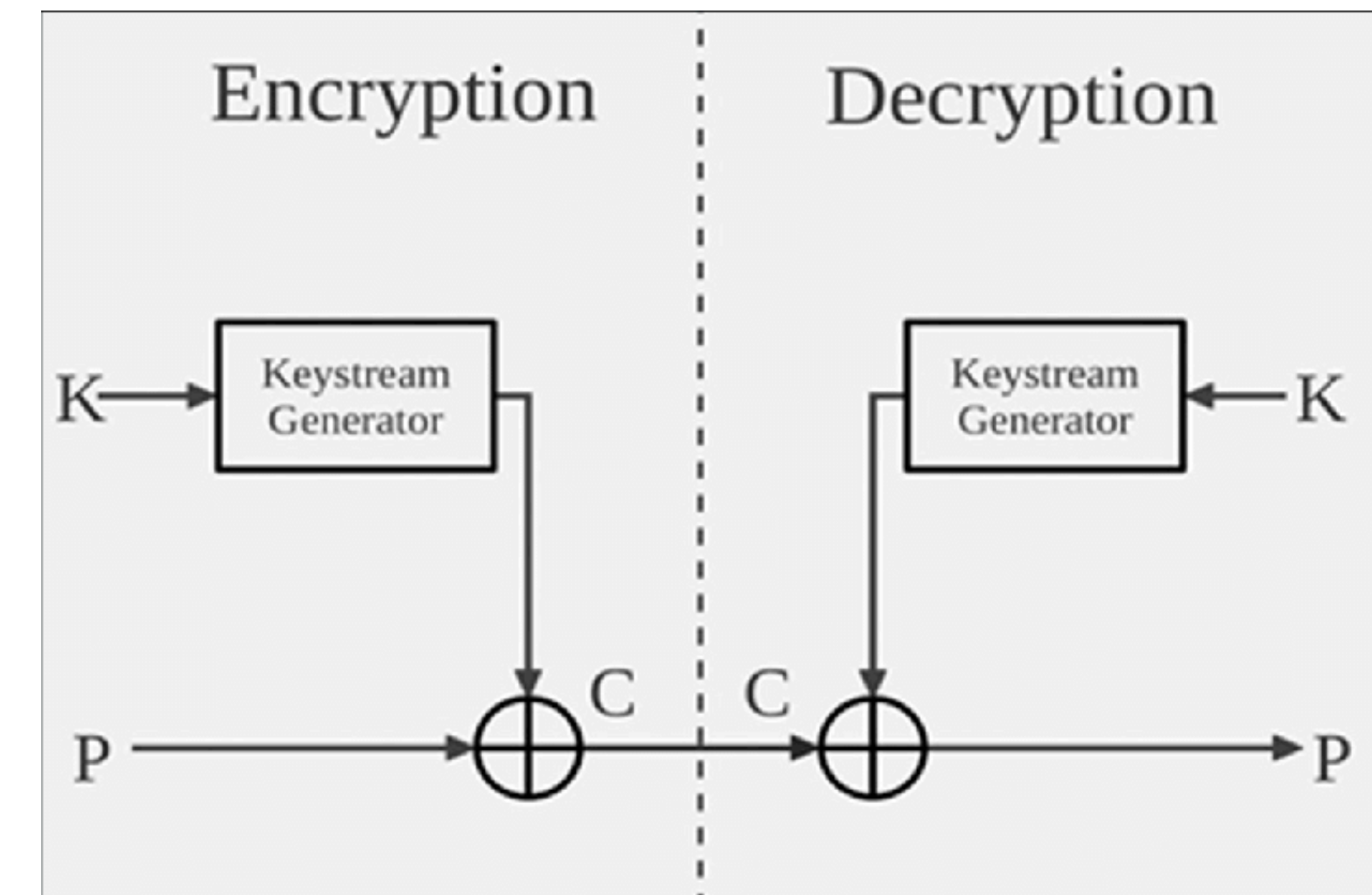
- Known to be perfectly (information-theoretically) secure



- Challenges for real-world applications
 1. Keys cannot be reused
 2. Truly random key with size greater or equal to the plaintext
 3. It has 0 diffusion (one message bit affects exactly one ciphertext bit)

Stream Ciphers

- **Basic Idea**
 - Generate an arbitrary-length keystream from a short, fixed-length key
 - Approximate OTP with the keystream
- The generation of the keystream must be **deterministic**
 - Therefore, the sequence must necessarily be **cyclic**
 - The **period** is the length of the sequence before it repeats



Stream Ciphers

Design criteria

- Period should be as long as possible (always longer than the plaintext)
- Key sequence must be:
 - **pseudo-random:** statistic properties of the sequence are those of a truly-random sequence
 - **unpredictable:** after observing a prefix of the sequence, it should be impossible to predict the “next bit”
- Other characteristics
 - **synchronous:** keystream is independent of the message
 - **self-synchronising:** able to recover synchronism when bits of the ciphertext are lost
 - **error-propagation:** the impact of transmission errors on decrypted messages
 - ...

Synchronous Stream Ciphers

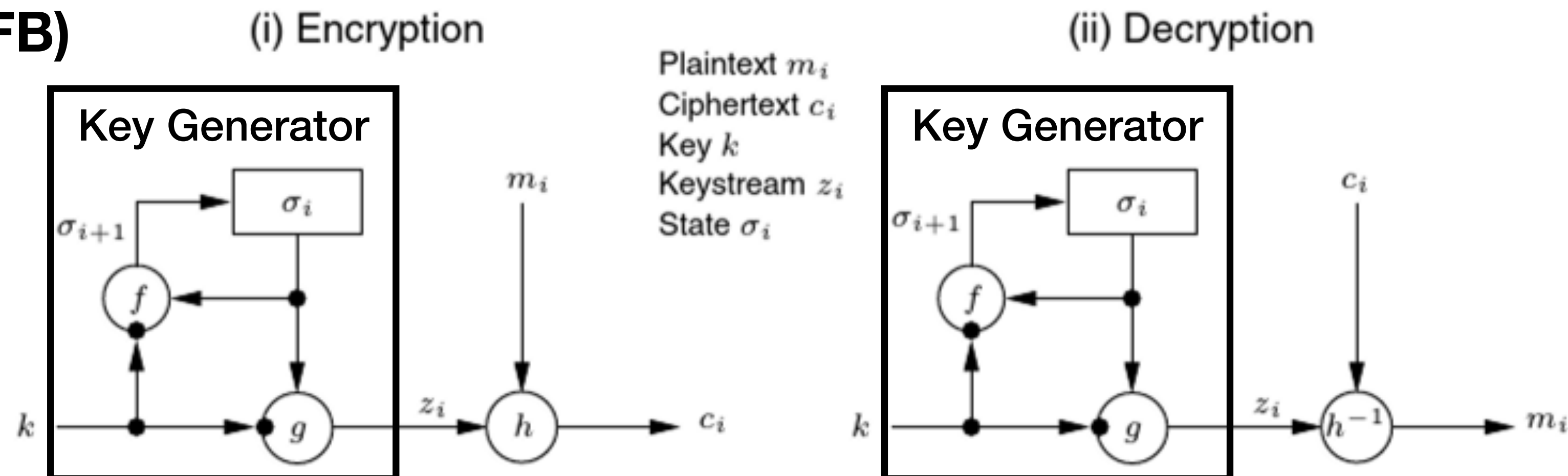
- Keystream is independent of the message
- Loss or insertion of bits in the cryptogram results in loss of synchronisation (decrypts garbage thereafter)
- Transmission errors (bit swapping) only affect the corresponding position of the original message
- The key affects

- **Output-Feedback Mode (OFB)**

- The next-state function f

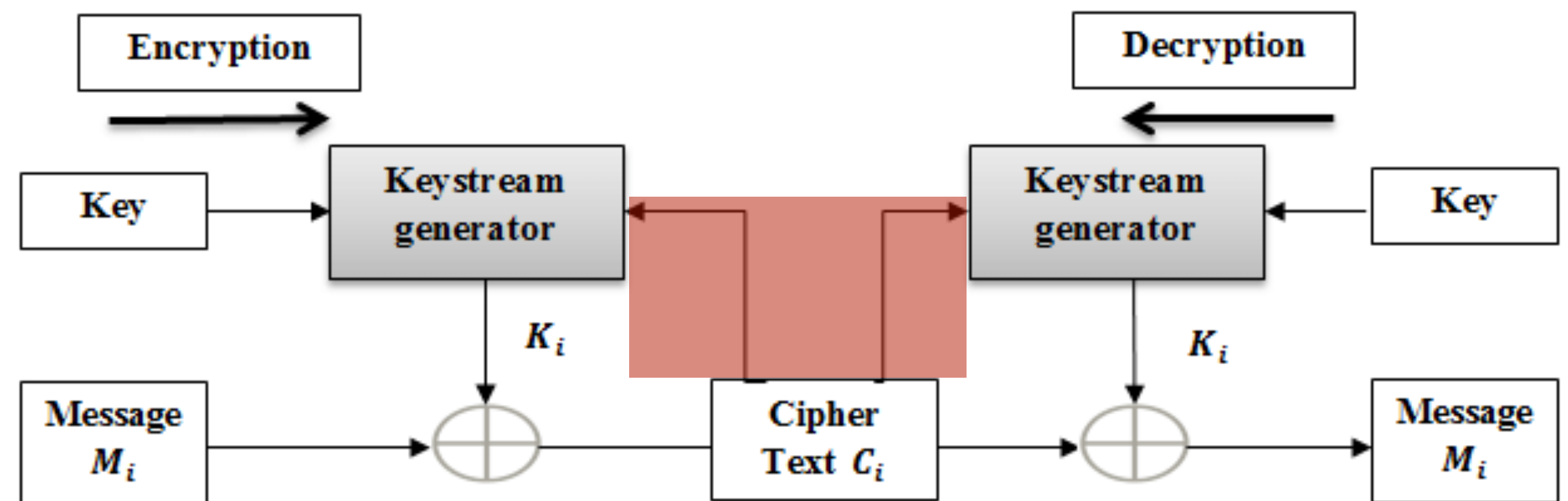
- **Counter Mode (CTR)**

- The output function g
- Both f and g



Self-synchronising Stream Ciphers

- Next bit is computed from the last n bits of the ciphertext (and key)
- If a bit is lost, synchronisation is restored once the “corrupted” bit is no longer used for the next bit computation
- Also known as **Cipher Feedback (CFB) Mode**
- Possible problem
 - Vulnerable to replay attacks
 - Cipher more tolerant to ciphertext manipulation



Stream Ciphers

- Stream ciphers are used in several well-known applications:
 - **A5 (A5/2)**: used in the GSM standard
 - **E0**: used in older versions of the Bluetooth protocol
 - **CSS (Content Scramble System)**: used to protect DVD discs

(All of the above examples are known to offer weak security guarantees)

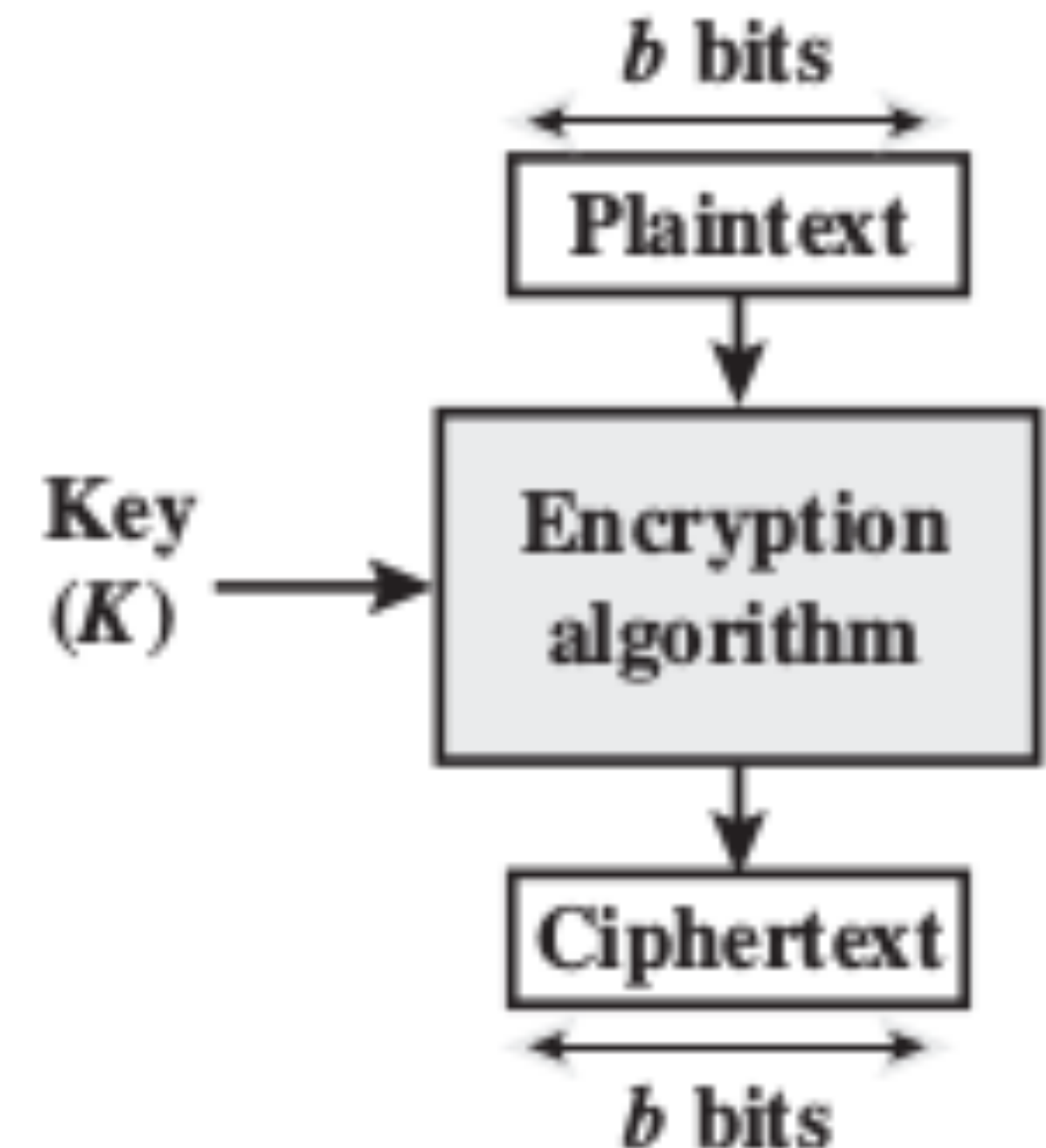
- **RC4 (ArcFour)**: a cipher designed for efficient software implementation, which was once widely used (e.g. WiFi) but **is now considered broken**
- **ChaCha20**: a modern and secure stream cipher
- Modern block ciphers (in specific modes) can also be turned into stream ciphers

NONCEs

- Stream ciphers so far have an essential but also problematic determinism property
 - The same key shall lead to the same generated keystream
 - Remember that the OTP precisely **forbids key(stream) reuse**
- This problem is generally overcome by using a **NONCE (Number used only ONCE)**
 - A number that should never be repeated
 - But not required to be kept secret (and not always random)
 - In practice, the Nonce is typically a sequence of randomly generated bytes used during encryption and sent along with the cryptogram

Block Ciphers

- A block cipher is a simpler primitive that processes fixed-length messages (a **block**)
- Can be generalized to a proper cipher (dealing with messages of arbitrary length) using standardized constructions (**modes of operation**)
- Security-wise, the primitive defines a permutation
 - For a randomly sampled k
 - The ciphertext must be indistinguishable from a pseudo-random permutation (PRP) (with negligible adversarial advantage)



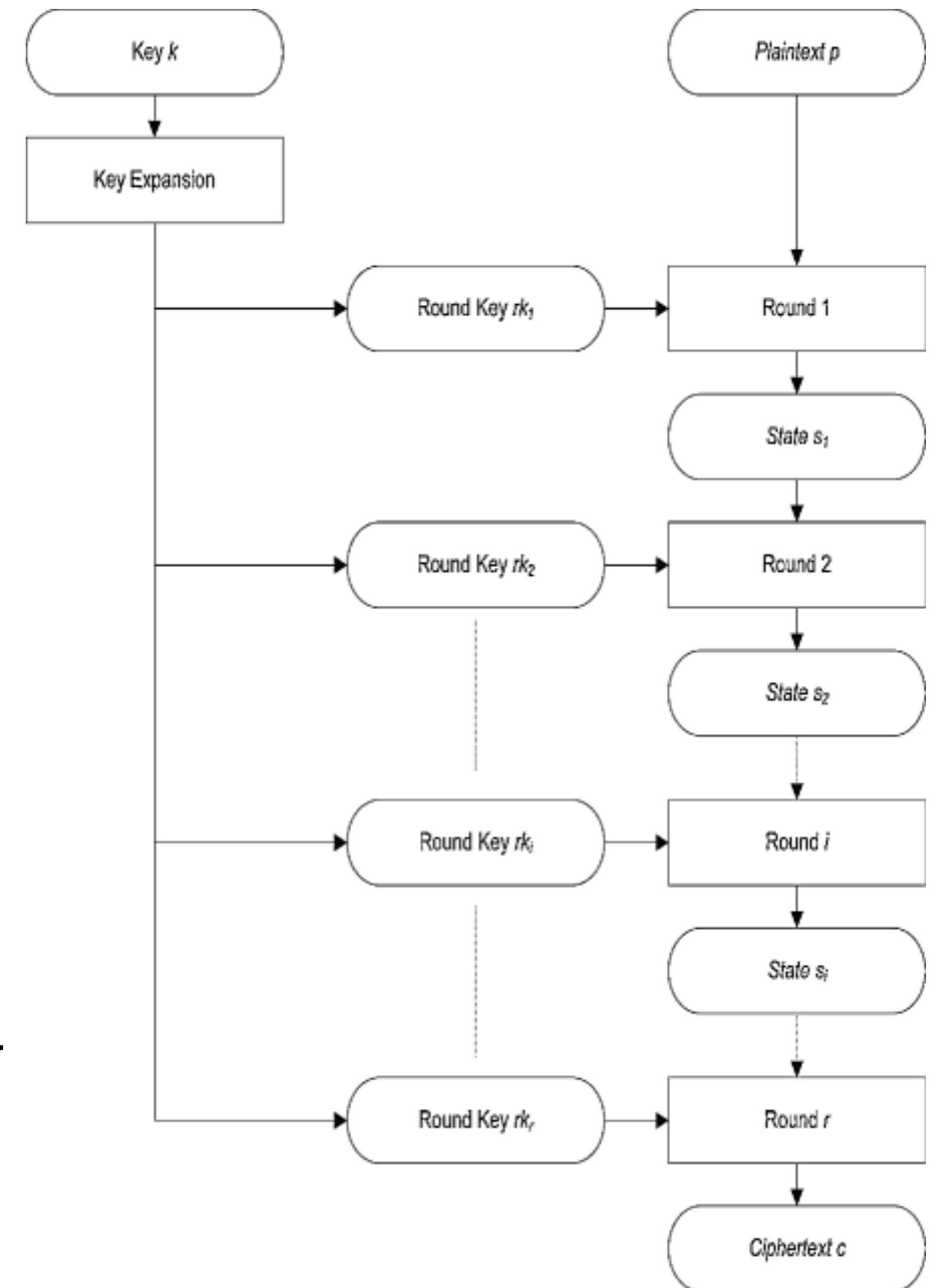
Block Ciphers

- A block cipher is defined by two **deterministic** algorithms (λ = key size, B = block size)
 - Encrypt: $E(k,p)$
 - Takes a key $k \in \{0, 1\}^\lambda$
 - Takes a plaintext $p \in \{0, 1\}^B$
 - Outputs a ciphertext $c \in \{0, 1\}^B$
 - Decrypt: $D(k,c)$
 - Takes a key $k \in \{0, 1\}^\lambda$
 - Takes a ciphertext $c \in \{0, 1\}^B$
 - Outputs a plaintext $p \in \{0, 1\}^B$
- A block cipher is **invertible**

Block Ciphers

Design criteria

- Two desirable properties [*Claude Shannon 1949*]:
 - **Diffusion:** influence of changes in the plaintext should be spread throughout the ciphertext
 - **Confusion:** relationship between the ciphertext and the encryption key should be as complex as possible
- Most modern designs are **iterated ciphers**, where a bijective **round** (depending on the key size) transformation is applied repeatedly with distinct **round keys** (derived from the main key)



Block Ciphers

vs Stream Ciphers

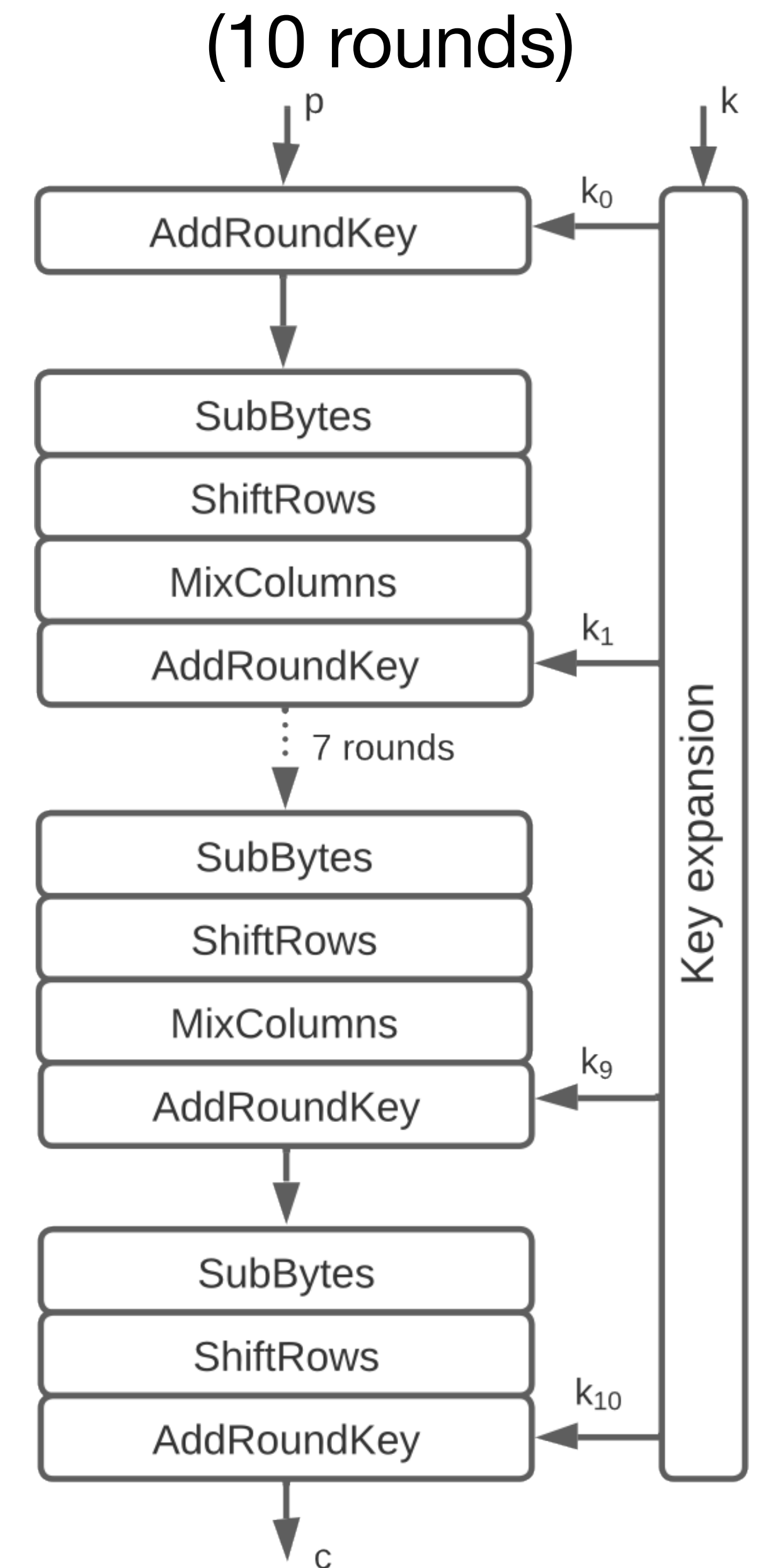
- Different “processing unit” (block vs bit/byte)
- Use of block ciphers depends on some mode of operation
- Stream ciphers do not promote diffusion
- Block cipher inverse operation (decryption) is harder to obtain
- Stream ciphers are typically faster than block ciphers
 - Often adopted for low-end devices

Advanced Encryption Standard (AES)

- Selected as a result of a NIST competition (1997-2000)
 - This process has since become the norm
 - NIST call for block cipher algorithm that would replace Data Encryption Standard (DES)
 - Criteria: performance and resistance to cryptanalysis
- Standardized in 2000
 - DES was still standard (56-bit keys)
 - 3DES was a common solution to improve security (use DES 3 times with 3 independent keys)
- Still the most used block cipher, by far
 - Available in mainstream CPUs as an hardware implementation

AES

- Block size 128 bits (16 bytes), organised as a 4x4 byte matrix
- Varying key size
 - 128 bits (10 rounds), 192 bits (12 rounds), 256 bits (14 rounds)
 - Expanded into required 128-bit round keys
- Features a detailed mathematical specification
- Internal state is transformed using a SP-network
- Round processing steps (with suitable inverses):
 - SubBytes, ShiftRows, MixColumns, AddRoundKey
- **Believed** to be resistant to all **known** attacks!

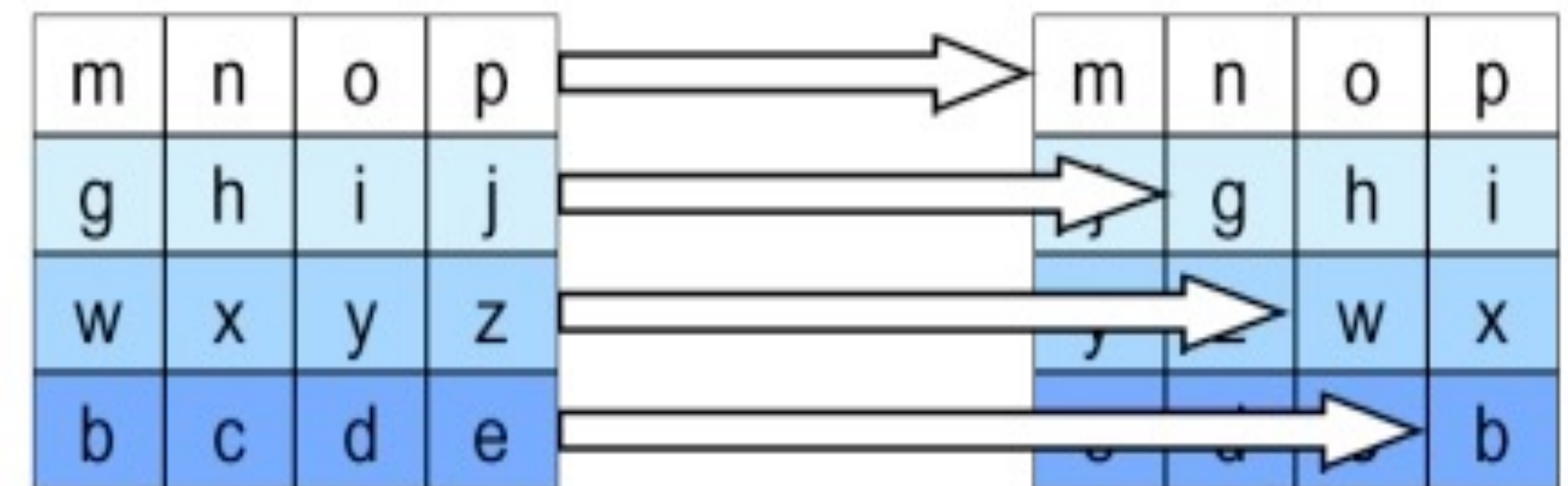
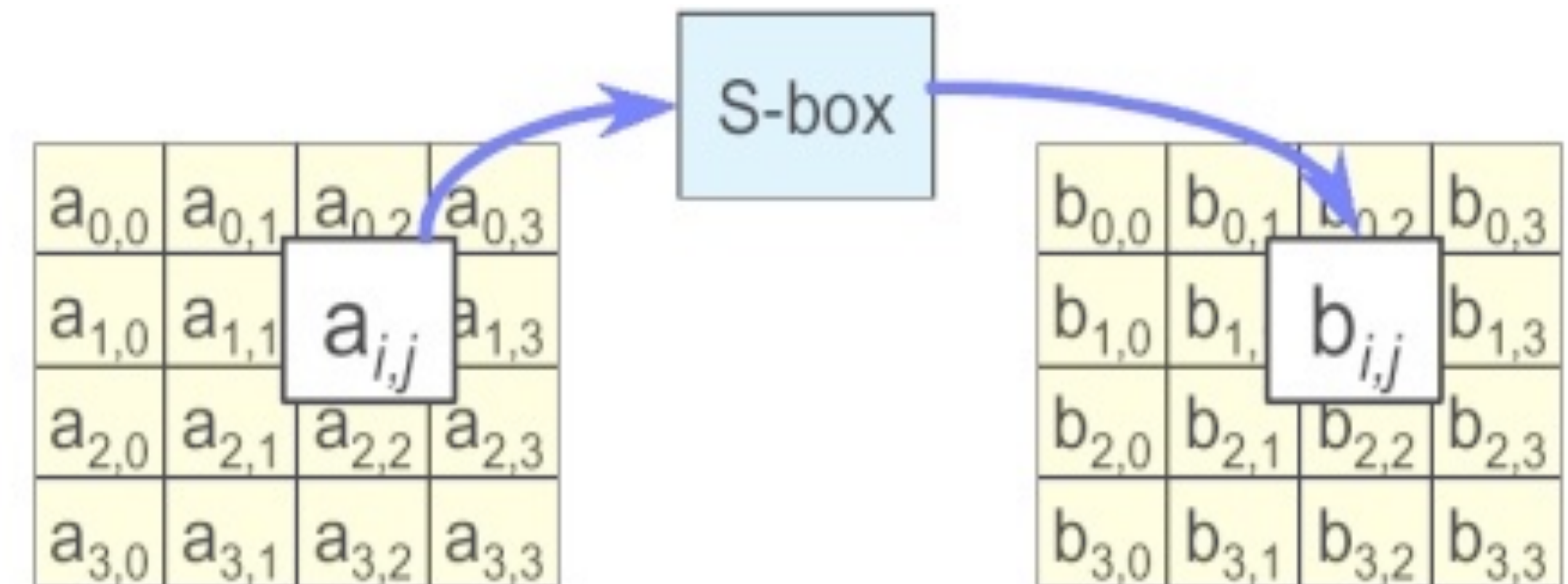


AES

Round steps

- SubBytes
 - Apply a single fixed S-box
 - Highly non-linear
 - The source of confusion
- ShiftRows
 - Shift each row by 0,1,2,3
 - Provides horizontal diffusion

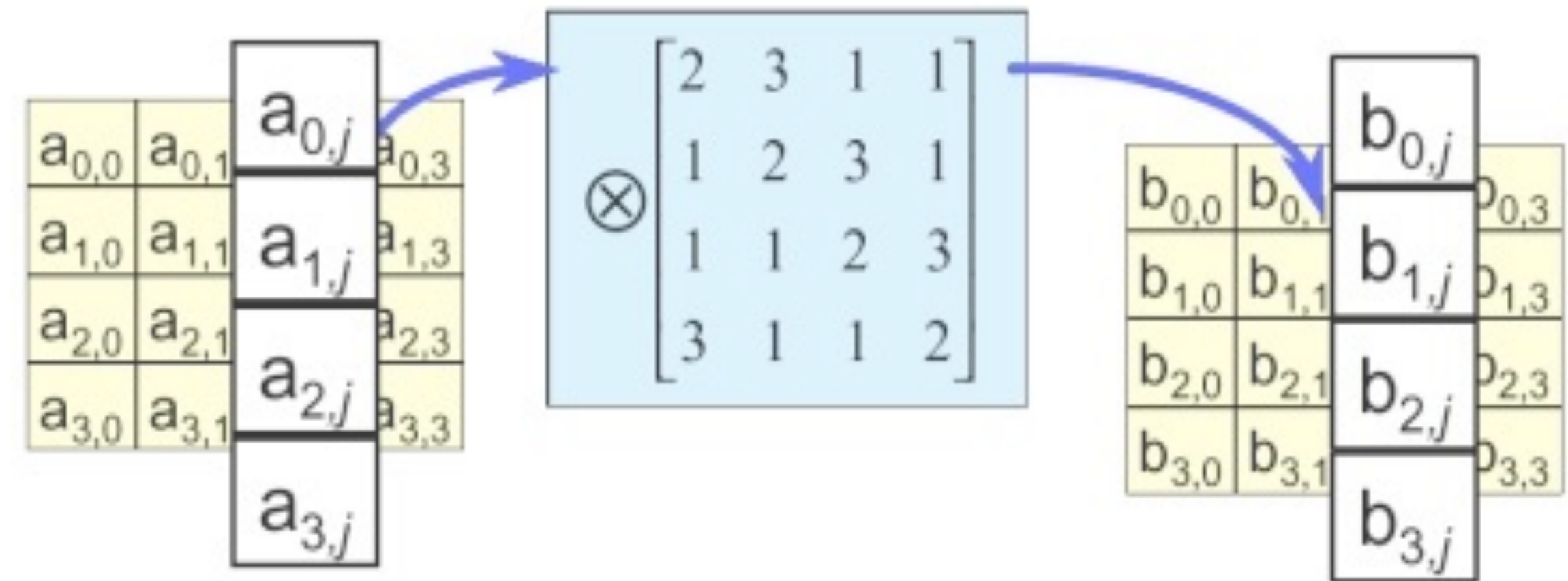
	Y															
X	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16



AES

Round steps

- MixColumns
 - Multiply by a fixed matrix
 - Provides vertical diffusion
 - Interaction with ShiftRows promotes high diffusion
- AddRoundKey
 - XOR with the round key
 - Only step dependent on the key



$$\begin{array}{|c|c|c|c|} \hline a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ \hline a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ \hline a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ \hline a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \\ \hline \end{array}
 +
 \begin{array}{|c|c|c|c|} \hline k_{0,0} & k_{0,1} & k_{0,2} & k_{0,3} \\ \hline k_{1,0} & k_{1,1} & k_{1,2} & k_{1,3} \\ \hline k_{2,0} & k_{2,1} & k_{2,2} & k_{2,3} \\ \hline k_{3,0} & k_{3,1} & k_{3,2} & k_{3,3} \\ \hline \end{array}
 =
 \begin{array}{|c|c|c|c|} \hline b_{0,0} & b_{0,1} & b_{0,2} & b_{0,3} \\ \hline b_{1,0} & b_{1,1} & b_{1,2} & b_{1,3} \\ \hline b_{2,0} & b_{2,1} & b_{2,2} & b_{2,3} \\ \hline b_{3,0} & b_{3,1} & b_{3,2} & b_{3,3} \\ \hline \end{array}$$

Block Ciphers

Modes of operation

- Block-ciphers are a **primitive**, they are not very useful on their own
 - There are **insecure** ways to encrypt with a block cipher
- A block cipher **mode of operation** describes how the primitive can be turned into an encryption scheme
 - We prove encryption secure assuming a block cipher is secure
- Depending on the application, we can choose among several standard modes of operation: ECB, CBC, CTR, ...
 - Each mode of operation comes equipped with its own security analysis: assumptions, proofs, etc

Block Cipher Modes

Padding

- Certain block cipher modes require the size of the plaintext to be a multiple of the block size
- Re-encode messages with a **padding** method (a reversible transformation that fills the last block of the plaintext)
 - Several standard constructions available
 - bit vs. byte oriented
 - deterministic vs. randomised
 - (Ciphertext Stealing is an alternative strategy)
 - E.g. PKCS7 padding
 - The last byte always reveals how much padding was added

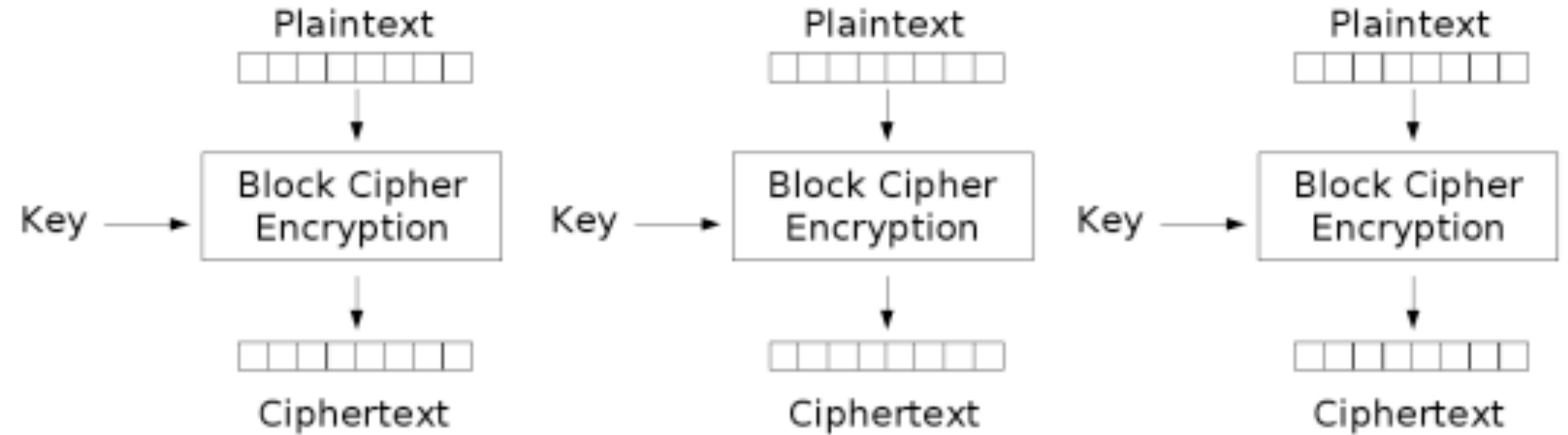
PKCS#7 Valid Padding

'A'	'B'	'C'	05	05	05	05	05
'A'	'B'	'C'	'D'	04	04	04	04
'A'	'B'	'C'	'D'	'E'	03	03	03
'A'	'B'	'C'	'D'	'E'	'F'	02	02
'A'	'B'	'C'	'D'	'E'	'F'	'G'	01
'A'	'B'	'C'	'D'	'E'	'F'	'G'	'H'
08	08	08	08	08	08	08	08

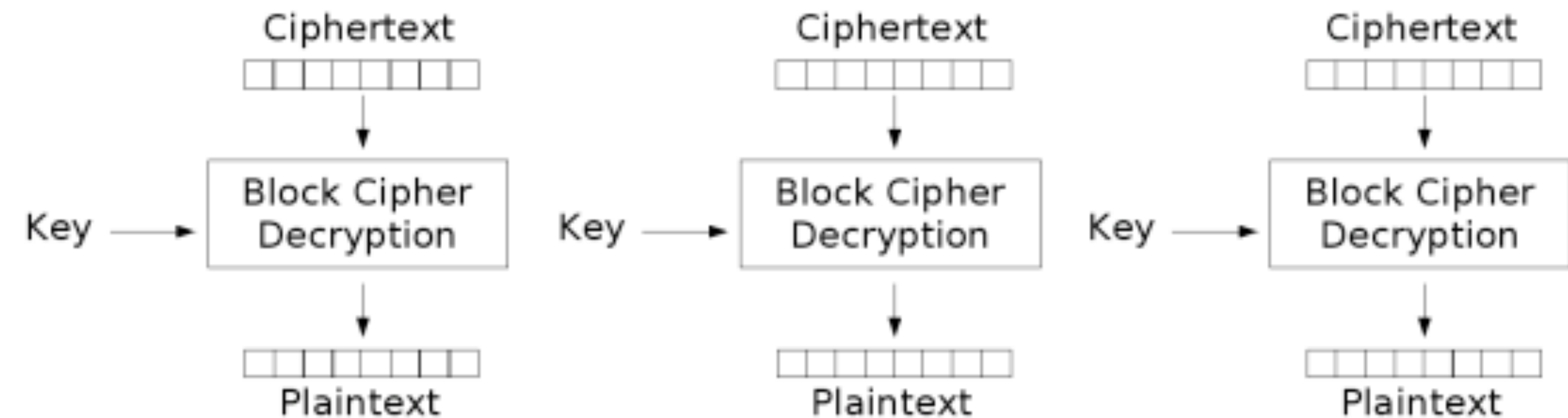
Block Cipher Modes

Electronic Code Book (ECB)

- Break message into plaintext blocks
- Apply padding to last block
- Independently encrypt each block
 - **Using the same key**
- Is this secure?



Electronic Codebook (ECB) mode encryption



Electronic Codebook (ECB) mode decryption

Block Cipher Modes

Electronic Code Book (ECB)

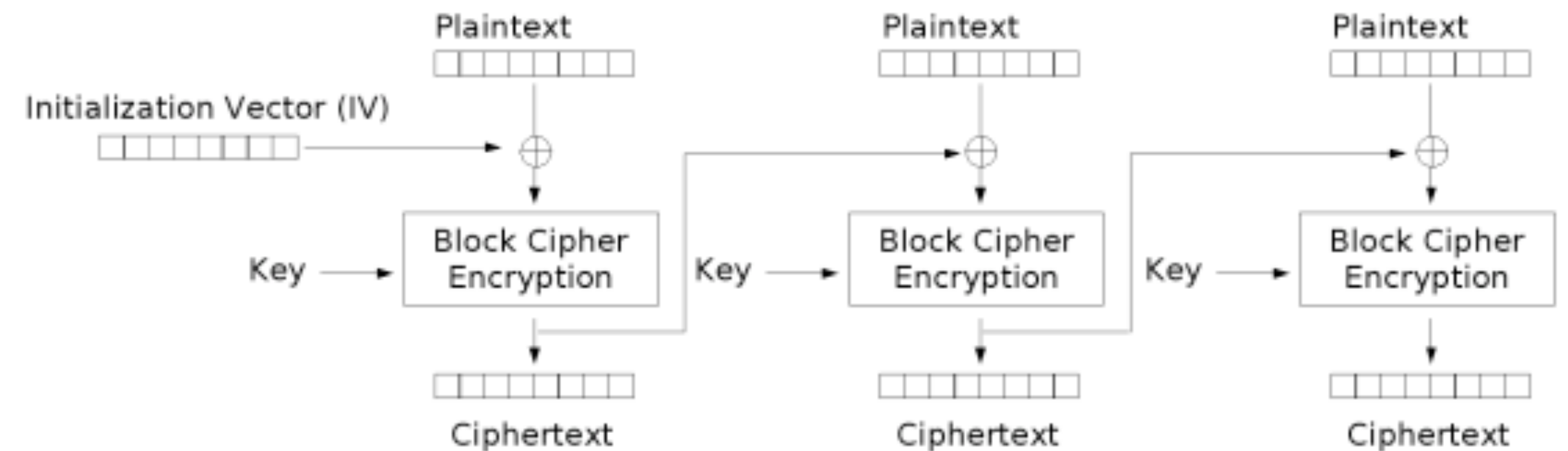
- Insecure to encrypt more than single-block messages
 - Leaks patterns of the plaintext
 - Vulnerable to repetition/replacement, known as **code book attack**



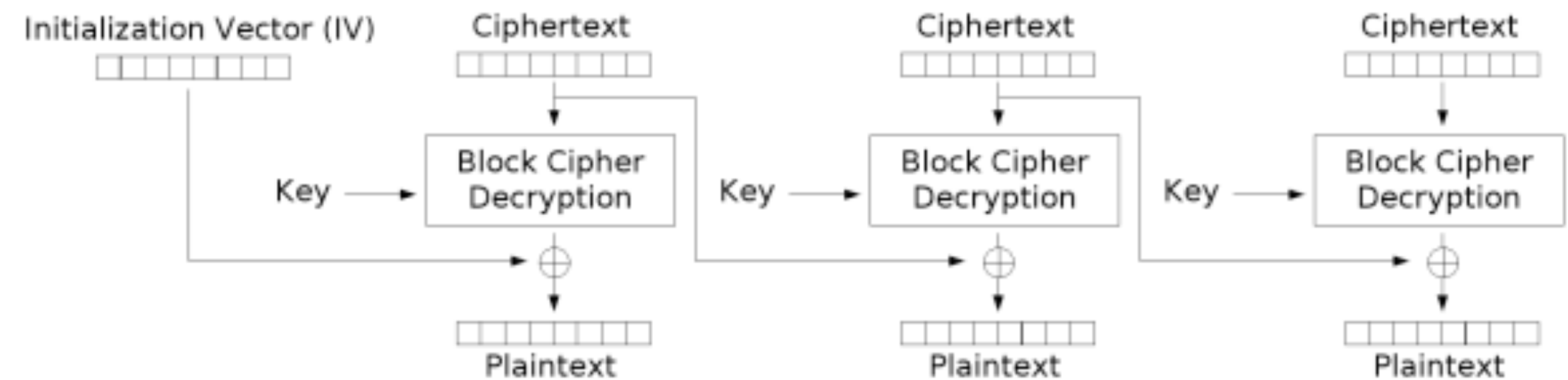
Block Cipher Modes

Cipher Block Chaining (CBC)

- Main difference to ECB is the Initialization Vector (IV)
 - Shall be random
- **Blocks now depend on each other**
 - Each plaintext block is masked by the previous ciphertext
- Is this secure?



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

Block Cipher Modes

Cipher Block Chaining (CBC)

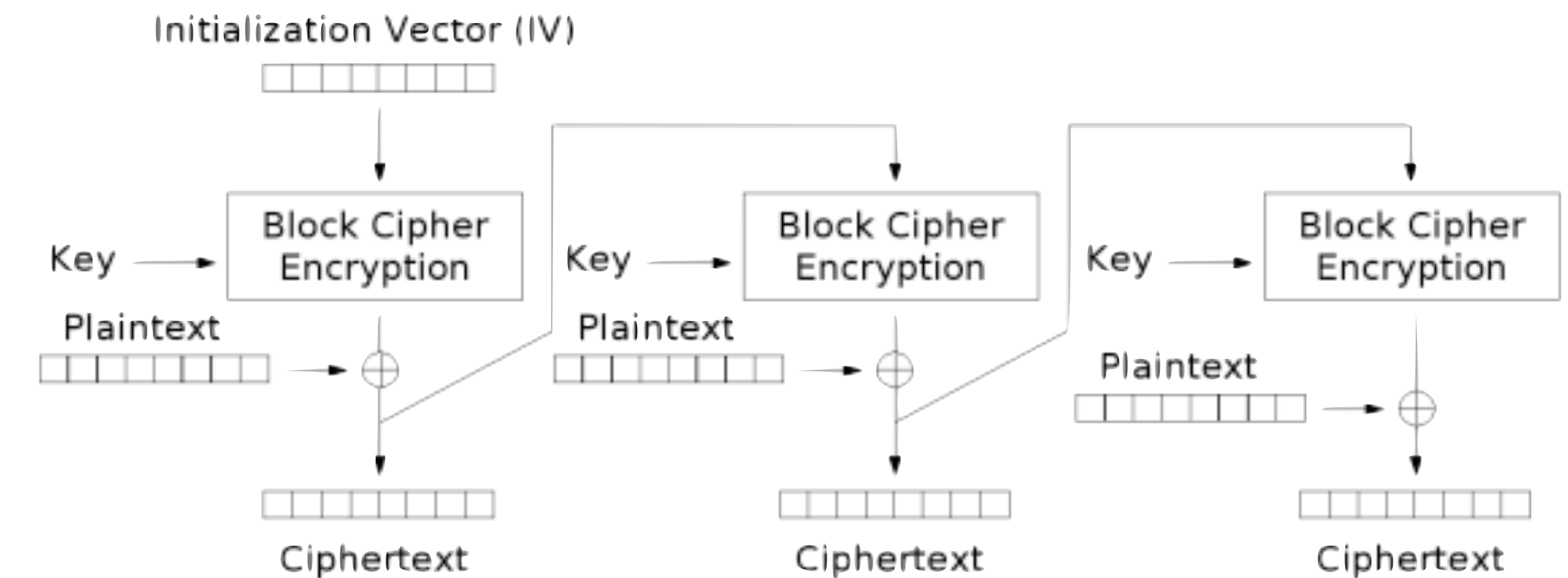
- **Secure to encrypt multi-block messages**
 - Block masking induces a dependency on all previous blocks
 - Changing one bit of the ciphertext impacts that block plus a single bit of the next-block
 - As with Nonces, the IV doesn't need to be kept secret



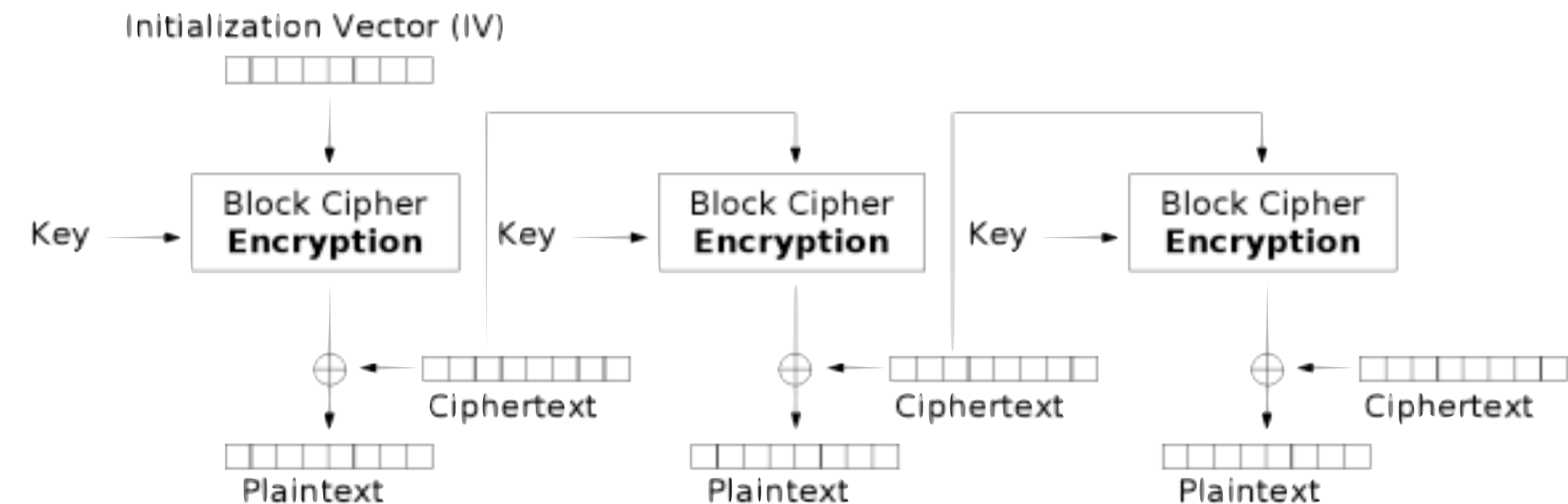
Block Cipher Modes

Cipher FeedBack mode (CFB)

- Implements a self-synchronising stream cipher
- Does not requires padding
- IV should be a Nonce
- **Also a “chain mode”**
 - The cipher encrypts the previous ciphertext block
- Uses Encryption to Decrypt



Cipher Feedback (CFB) mode encryption



Cipher Feedback (CFB) mode decryption

Block Cipher Modes

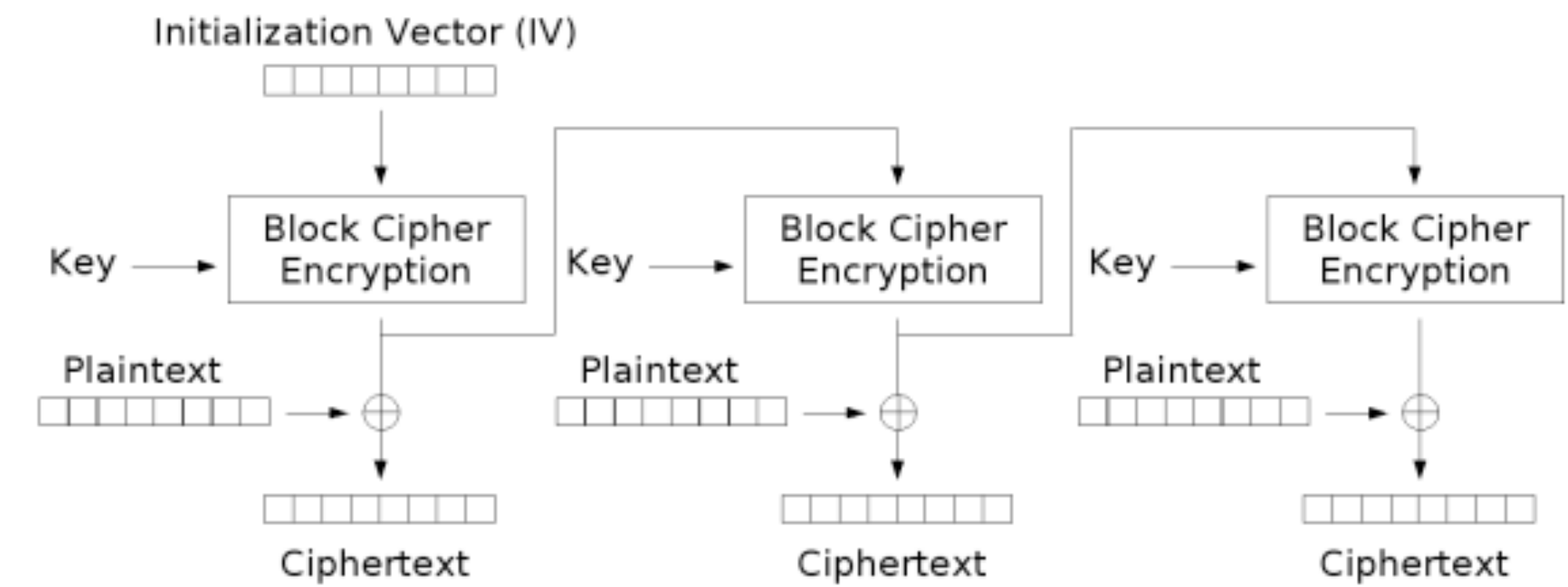
Cipher FeedBack mode (CFB)

- Notice that the block cipher is always used in “encrypt” mode
 - It is not encrypting the message, but producing a keystream
- The cipher is **sequential**
 - Keystream depends on the key, IV, and all the previous plaintext
- Supports **feedback** (reusing n bits of the ciphertext from the previous block)
 - n between 1 and the block size
 - Affects the resynchronisation time
- Changing a bit in the ciphertext affects the current block plus one bit (similar to CBC)

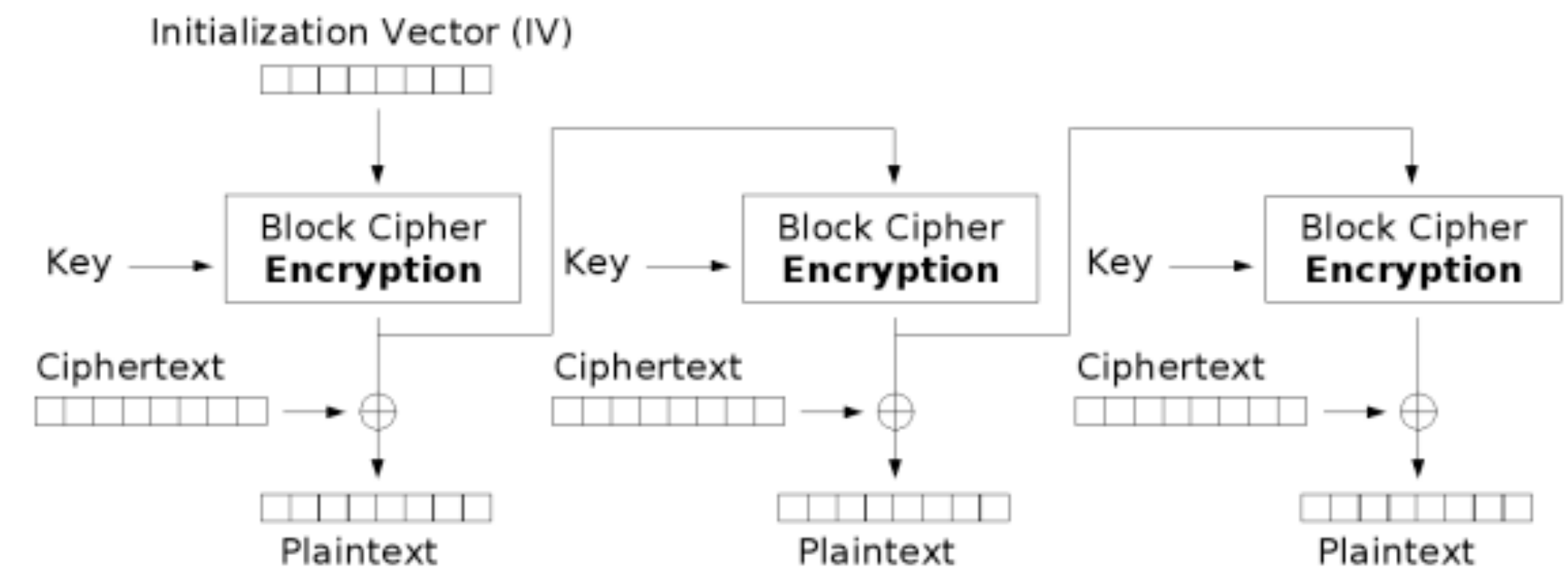
Block Cipher Modes

Output FeedBack mode (OFB)

- Implements a synchronous stream cipher (in output-feedback mode)
- Does not requires padding
- IV should be a Nonce
- **Also a “chain mode”**
 - The cipher encrypts its own previous output
- Uses Encryption to Decrypt



Output Feedback (OFB) mode encryption



Output Feedback (OFB) mode decryption

Block Cipher Modes

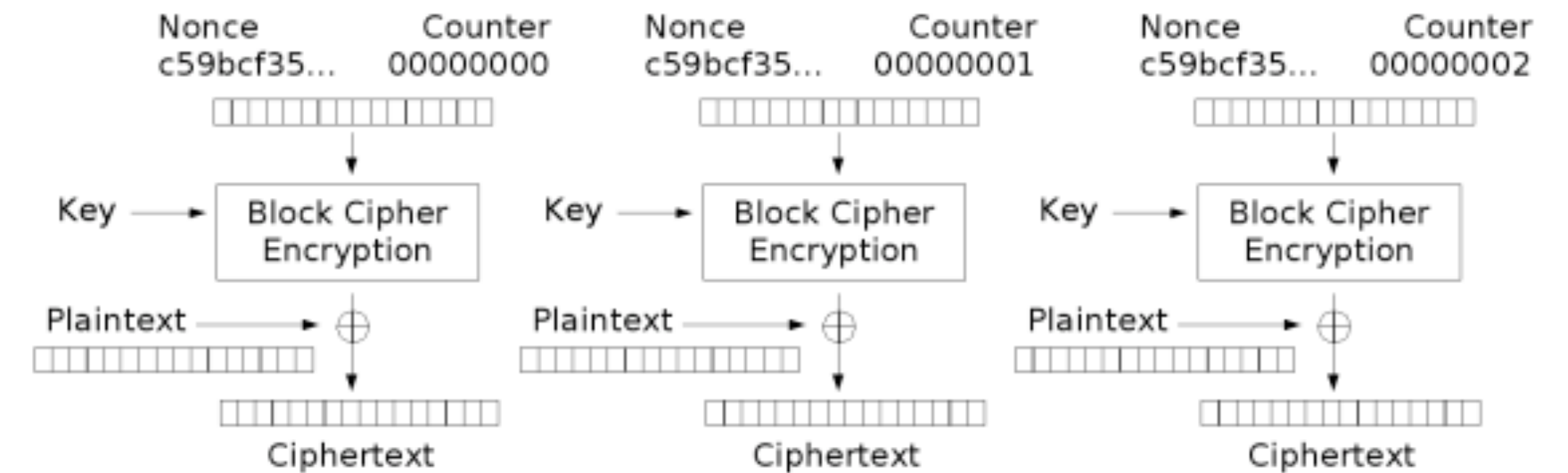
Output FeedBack mode (OFB)

- The block cipher is used as the *next-state function*
- The *output-function* is the identity function
- The cipher is (slightly less) sequential
 - Keystream iterates the block cipher on the IV
 - Keystream does not depend on the message
 - It can be expanded prior to the reception of the message
- Changing one bit of the ciphertext only impacts a single bit

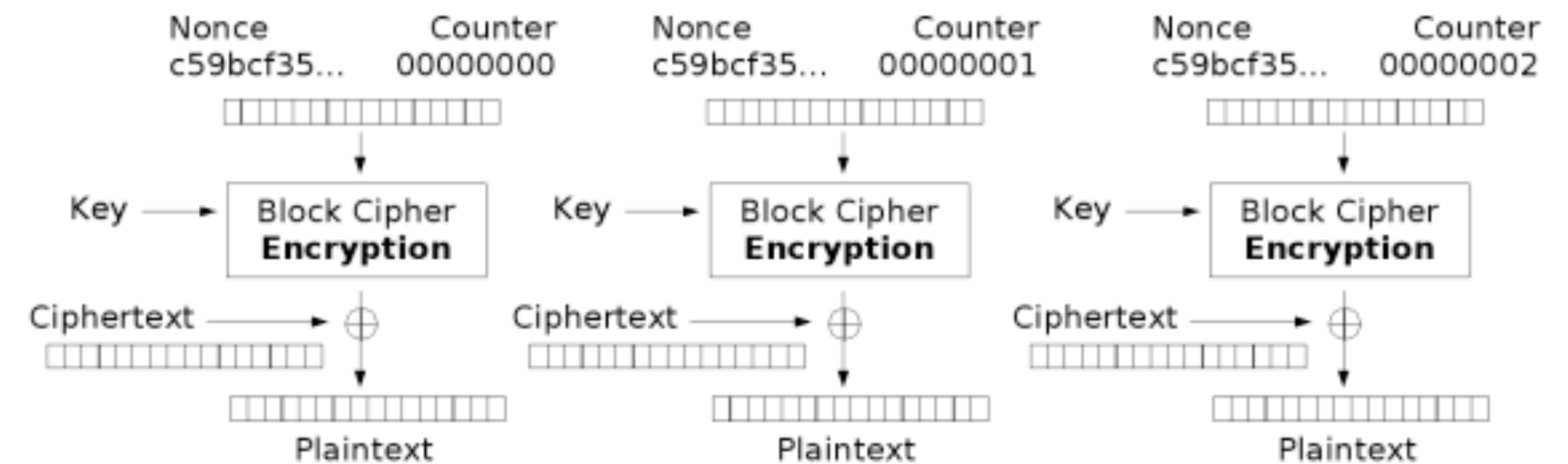
Block Cipher Modes

CounTeR mode (CTR)

- Implements a synchronous stream cipher (in counter mode)
- Does not requires padding
- IV should be a Nonce
- **A “parallel” mode**
 - The cipher encrypts a predictable counter
- Uses Encryption to Decrypt



Counter (CTR) mode encryption



Counter (CTR) mode decryption

Block Cipher Modes

CounTeR mode (CTR)

- The *next-state function* is a simple increment
- The block cipher is used as the *output-function*
- The cipher is parallel
 - Keystream does not depend on the message
 - It can be expanded prior to the reception of the message
 - Keystream blocks use a fixed counter
 - Each block of the keystream can be processed independently
- Changing one bit of the ciphertext only impacts a single bit

On Malleability

- We have seen two properties of ciphers
 - **Confusion:** a change in one bit of the key shall unpredictably change as most of the ciphertext as possible
 - Prevents attacks that attempt to recover the key
 - **Error-propagation:** how much the (accidental) change of one bit of the ciphertext affects the plaintext decryption
 - A trade-off in robustness / security, e.g., synchronous stream ciphers
- But encryption schemes may still be vulnerable to attacks
 - **Malleability:** how predictable is the (intentional) change of one bit of the ciphertext on the plaintext
 - E.g., the OTP has 0 error-propagation and high malleability (an attacker can flip bits in the ciphertext to change one bit of the plaintext)
 - Error-propagation and malleability are typically inversely correlated

On Malleability

Authenticated Encryption



- We can stop malleability using **Authenticated Encryption**
 - We will see MACs (Message Authentication Codes) and AEAD (Authenticated Encryption with Associated Data) AEAD
- A recent trend has been the adoption of block cipher modes of operation which provide both **confidentiality** and **authenticity/integrity** of data
 - Some examples:
 - **EAX** (Encrypt-then-Mac-then-Translate)
 - **CCM** (Counter with CBC-MAC)
 - **GCM** (Galois/Counter Mode)
 - **OCB** (Offset CodeBook)